· Consider the “lost” points as outliers (not really reliable, though, as they might lie in the middle of some big cluster that can’t be covered by a single canopy);
· Enlarge canopies: the discarded centroids can be appropriately marked during a run of canopy clustering, every time one of the centroids is chosen. During this phase, it is possible to keep track of the outer radius associated with each canopy, and make it large enough to cover all points in the canopy of the removed point: the cheapest way to do so is to set the radius of all canopies to T_1+T_2, but this would obviously be increasing the overlapping between canopies;
· Go through unassigned points (those that are not within a distance T_1 from any of the survived centroids) at the end of the classification step and assign them to the closest centroid. This solution will limit to the minimum the overlapping of canopies (each canopy’s radius will be at most T_1+T_2, but as small as the distance to the furthest of these new points), but it will be more costly.
· If we perform classification in the same MapReduce job as the choice of centroids, we can find an efficient solution: mappers will have to produce also a list with the *sets* of points associated to each centroid, and pass these sets to the reducer along with the list of centroids for each shard; in the reducer, an ad-hoc variant of canopy clustering is run and when a centroid c is drawn, the sets assigned to all centroids within a radius T_2 from c are merged and assigned to c’s canopy. This is the best solution in terms of performance, because it saves one iteration over all points in all shards, and only require a single reducer. The downside is that it